

# Connection Pooling — HikariCP

A simple mental model:

Without pooling:

```
Request
↓
Create DB connection
↓
Execute SQL
↓
Close connection
```

With pooling:

```
Application
↓
HikariCP Pool
├──
├──
├──
└──
↓ ↓ ↓ ↓
C1 C2 C3 C4
    |
    ↓
Database
```

The application reuses existing database connections instead of creating a new physical connection for every request.

---

## 1. What Is a Database Connection?

A database connection is a communication channel between your application and the DB server.

Creating one involves work such as:

```
TCP connection
↓
authentication
↓
session initialization
↓
database/network setup
```

That is relatively expensive compared with executing a simple SQL statement.

Imagine:

```
1000 requests
```

and every request creates a fresh DB connection:

```
1000 connection creations
```

```
+  
1000 connection closes
```

That's wasteful.

---

## 2. What Is Connection Pooling?

A **connection pool** creates a set of database connections ahead of time and keeps them available for reuse.

For example:

```
HikariCP  
-----  
Connection 1  
Connection 2  
Connection 3  
...  
Connection 20
```

When a request needs the DB:

```
Application  
↓  
Borrow connection  
↓  
Execute SQL  
↓  
Return connection
```

The important point:

*`Connection.close()` usually returns the connection to the pool rather than physically destroying the underlying database connection.*

Conceptually:

```
getConnection()  
↓  
borrow from pool  
↓  
use  
↓  
close()  
↓  
return to pool
```

---

## 3. What Is HikariCP?

**HikariCP** is a JDBC connection pool.

Spring Boot commonly uses HikariCP as the default JDBC connection pool when using the standard JDBC/JPA starters.

So your typical stack is:

```
Spring Boot
↓
DataSource
↓
HikariCP
↓
JDBC Connection
↓
Database
```

Then:

```
Spring Data JPA
↓
Hibernate
↓
JDBC
↓
HikariCP
```

This is important:

*Hibernate does not normally create physical DB connections directly for every query. It obtains JDBC connections through the configured `DataSource`, which in a typical Spring Boot setup is backed by HikariCP.*

---

## 4. What Happens During a JPA Query?

Suppose:

```
paymentRepository.findById(100L);
```

The simplified flow is:

```
Repository
↓
JPA
↓
Hibernate
↓
DataSource
↓
HikariCP
↓
Borrow JDBC connection
↓
Execute SQL
↓
Return connection
```

SQL:

```
SELECT *
FROM payment
WHERE id = ?;
```

---

After Hibernate is finished with the JDBC work, the connection is returned to the pool.

---

## 5. Why Do We Need a Pool?

---

Main benefits:

### Performance

Avoid repeatedly creating physical DB connections.

### Controlled concurrency

Suppose your application has:

```
500 concurrent requests
```

but the DB should only handle:

```
20 concurrent DB operations
```

The pool can limit active connections.

### Resource management

Without pooling, connection creation can become uncontrolled.

### Reuse

Connections are reused instead of recreated.

---

## 6. Pool Size $\neq$ Application Thread Count

---

Very important.

Suppose:

```
Application threads = 200  
Hikari pool = 20
```

You can have:

```
200 application tasks  
    ↓  
20 DB connections  
    ↓  
DB
```

Only 20 tasks can actively hold a DB connection at a time.

The others wait until a connection becomes available.

So:

---

Threads  
≠  
DB connections

---

## 7. What Happens When the Pool Is Exhausted?

---

Suppose:

Pool size = 10

and:

10 requests

are currently holding connections.

Request 11 needs a connection:

```
Request 11
  ↓
HikariCP
  ↓
No connection available
  ↓
Wait
```

The request waits up to the configured **connection acquisition timeout**.

If no connection becomes available within that period, HikariCP fails the acquisition.

This is a key operational failure mode:

```
Application concurrency
  ↓
Too many DB users
  ↓
Pool exhausted
  ↓
Requests wait
  ↓
Timeouts
  ↓
Latency / errors
```

---

## 8. Important HikariCP Settings

---

You should know these concepts.

**maximumPoolSize**

Maximum number of connections in the pool.

Example:

---

```
spring.datasource.hikari.maximum-pool-size=20
```

Conceptually:

```
At most 20 connections
```

---

### minimumIdle

How many idle connections the pool tries to maintain, depending on its configuration/management behavior.

Example:

```
spring.datasource.hikari.minimum-idle=5
```

Don't interpret this as:

*"Exactly 5 connections always exist."*

Pool lifecycle behavior can change as connections are created/retired.

---

### connectionTimeout

Maximum time to wait for a connection from the pool.

Example:

```
spring.datasource.hikari.connection-timeout=30000
```

This is approximately:

```
30 seconds waiting for pool connection
```

Important distinction:

*This is **not** the same as a SQL execution timeout.*

---

### idleTimeout

Controls how long idle connections can remain before being eligible for retirement, subject to pool configuration.

```
spring.datasource.hikari.idle-timeout=600000
```

---

## maxLifetime

Maximum lifetime of a pooled connection before Hikari retires it.

```
spring.datasource.hikari.max-lifetime=1800000
```

Why?

Infrastructure such as:

- database
- load balancer
- firewall
- network infrastructure

may terminate long-lived connections.

You generally want the pool's lifetime to be compatible with the infrastructure's connection lifetime policies.

---

## LeakDetectionThreshold

Can help detect connections that appear to remain borrowed too long.

```
spring.datasource.hikari.leak-detection-threshold=60000
```

This is a **diagnostic feature**, not a way to solve connection leaks.

---

## 9. Connection Timeout vs Query Timeout

Very common interview question.

### Connection acquisition timeout

```
How long do I wait to get a connection from Hikari?
```

### Query/socket timeout

```
Once I have the connection, how long should the database/network operation be allowed to run?
```

These are different.

```
Request
↓
Wait for Hikari connection
↓
Get connection
↓
Execute SQL
↓
DB/network timeout
```

---

## 10. Connection Pool Exhaustion

---

A very common production scenario.

Suppose:

```
Pool size = 20
```

All 20 connections are occupied.

Why might this happen?

### Slow SQL

```
Query takes 30 seconds
```

### Long transactions

```
Transaction starts
  ↓
Connection held
  ↓
Many unrelated operations
  ↓
Transaction finally commits
```

### Connection leak

Application obtains a connection and fails to return it properly.

### Too much concurrency

Huge request volume can consume all available DB connections.

### External API inside DB transaction

This is an especially important backend mistake:

```
@Transactional
public void process() {

    updateDatabase();

    callExternalApi(); // 5 seconds

    updateDatabase();
}
```

The DB connection may remain associated with the transaction while the application waits on the external API, depending on the transaction/persistence behavior.

That means:

```
DB connection
  ↓
waiting on HTTP call
  ↓
```

```
not doing DB work
↓
but unavailable to other requests
```

This can starve the pool.

---

## 11. Why Pool Size Shouldn't Be Huge

---

A common misconception:

```
"More connections = more performance."
```

Not necessarily.

Suppose:

```
Hikari pool = 200
```

but your database can efficiently process only a limited number of concurrent queries.

Now you may get:

```
200 concurrent queries
↓
DB contention
↓
CPU pressure
↓
memory pressure
↓
I/O contention
↓
slower queries
```

So increasing the pool can actually make the system worse.

Connection-pool sizing should consider:

- DB capacity
- CPU
- workload
- query latency
- transaction duration
- number of application instances
- concurrency
- DB connection limits

---

## 12. Pool Size in a Microservice Environment

---

This is an excellent interview scenario.

Suppose:

10 application instances

and each instance has:

maximumPoolSize = 20

Potential DB connections:

$10 \times 20 = 200$

The database doesn't see:

"20 connections"

It can see up to:

"200 connections"

So pool sizing must consider **all application instances collectively**.

This is very important in Kubernetes/microservice environments.

---

## 13. HikariCP and Hibernate

---

Think:

```
Spring Data JPA
  ↓
Hibernate
  ↓
JPA EntityManager
  ↓
JDBC
  ↓
DataSource
  ↓
HikariCP
  ↓
Database
```

Hibernate handles things like:

```
ORM
SQL generation
persistence context
dirty checking
entity lifecycle
```

Hikari handles:

```
JDBC connection pooling
connection acquisition
connection reuse
connection lifecycle
pool limits
```

So don't say:

*"Hikari manages Hibernate entities."*

It manages **JDBC connections**.

---

## 14. Spring Boot Configuration Example

---

Typical configuration:

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/paymentdb
    username: appuser
    password: secret

  hikari:
    maximum-pool-size: 20
    minimum-idle: 5
    connection-timeout: 30000
    idle-timeout: 600000
    max-lifetime: 1800000
```

You don't need to memorize these numbers.

The important thing is knowing **what each setting controls**.

---

## 15. HikariCP + @Transactional

---

Consider:

```
@Transactional
public void processPayment() {

    Payment payment =
        repository.findById(100L)
            .orElseThrow();

    payment.setStatus("PAID");

    historyRepository.save(history);
}
```

Conceptually:

```
Transaction begins
↓
Hibernate needs DB connection
↓
Hikari provides connection
↓
SELECT
↓
UPDATE
↓
INSERT
↓
```

```
Flush
↓
Commit
↓
Connection returned
```

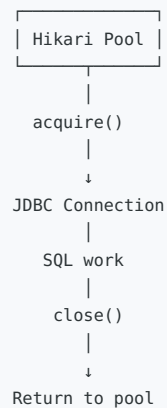
The pool is therefore part of the infrastructure supporting your transaction.

---

## 16. Connection Lifecycle

---

Simplified:



The important trick:

```
connection.close();
```

does not necessarily mean:

```
Destroy physical DB connection
```

With a pool-backed `DataSource`, `close` generally returns the logical connection to the pool.

---

## 17. Connection Leak

---

A connection leak occurs when an application borrows a connection but fails to return it.

Bad JDBC code could look like:

```
Connection connection =
    dataSource.getConnection();

doSomething();

// exception occurs
// connection never closed
```

Correct:

```
try (Connection connection =
```

```
dataSource.getConnection() {  
  
    doSomething();  
}
```

The try-with-resources ensures cleanup.

With Spring Data JPA/Hibernate, you normally don't manually manage connections for ordinary repository operations. Spring/Hibernate manage the JDBC resource lifecycle around transactions.

---

## 18. Leak Detection

Hikari can help identify connections that remain borrowed for an unusually long time.

For example:

```
spring.datasource.hikari.leak-detection-threshold=60000
```

If a borrowed connection remains out longer than the configured threshold, Hikari can log diagnostic information.

But:

*A long-held connection is not necessarily a leak.*

It could be:

```
slow legitimate transaction
```

So treat leak detection logs as **investigation signals**, not automatic proof of a bug.

---

## 19. Pool Metrics

For production systems, you should monitor things such as:

```
Active connections  
Idle connections  
Total connections  
Pending/waiting threads  
Connection acquisition latency  
Connection timeout count
```

Conceptually:

```
Pool:  
Maximum = 20  
  
Active = 20  
Idle   = 0  
Waiting = 50
```

That's a warning sign that DB concurrency is saturated.

Spring Boot Actuator/Micrometer can expose application/database pool metrics depending on your configuration.

---

## 20. HikariCP and Observability

---

Your production diagnosis could look like:

```
API latency increased
↓
Check application metrics
↓
DB connection pool active = 20/20
↓
Many threads waiting
↓
Check DB
↓
Slow query / lock contention
↓
Fix root cause
```

Notice:

*Increasing the pool isn't necessarily the first fix.*

The real problem may be:

```
slow query
long transaction
DB lock
connection leak
external call inside transaction
```

---

## 21. Connection Pool vs Thread Pool

---

Very common interview question.

### Thread pool

Controls:

How many tasks/threads execute concurrently?

Example:

ExecutorService

### Connection pool

Controls:

---

How many DB connections can be borrowed concurrently?

Example:

HikariCP

They are independent resources.

```
200 application threads
      ↓
20 DB connections
      ↓
Database
```

A system may have many threads but a much smaller DB pool.

---

## 22. Connection Pool vs HTTP Connection Pool

Also don't confuse:

```
HikariCP
  → JDBC/database connections

HTTP connection pool
  → HTTP client connections
```

For example:

```
BPS
├─ HikariCP → database
└─ HTTP client pool → Hydra / Chase
```

They solve similar resource-reuse problems but for different protocols.

---

## 23. Connection Pooling vs Database Transactions

These are also different.

### Pooling

Answers:

*How do we efficiently manage DB connections?*

### Transaction

Answers:

*Which database operations succeed or roll back together?*

They interact:

```
Transaction
  ↓
uses DB connection
  ↓
connection acquired from Hikari
```

But:

```
Connection pool ≠ transaction manager
```

---

## 24. Connection Pooling vs Prepared Statement Cache

Another distinction.

Hikari manages:

```
Connections
```

Some JDBC drivers/database configurations may separately manage or cache:

```
Prepared statements
```

Don't attribute every JDBC performance optimization to Hikari.

---

## 25. Practical Failure Scenario

Imagine:

```
maximumPoolSize = 10
```

Then:

```
Request 1 → connection
Request 2 → connection
...
Request 10 → connection
```

All 10 are busy.

Now:

```
Request 11
Request 12
...
Request 100
```

need DB access.

They wait for connections.

If the first 10 requests are slow because of:

```
long-running SQL
```

then eventually:

```
connectionTimeout
```

occurs.

Result:

```
DB pool exhaustion
  ↓
request latency
  ↓
connection timeout
  ↓
HTTP errors
```

Now imagine the application retries those failed requests.

You can make things worse:

```
timeouts
  ↓
retries
  ↓
more DB requests
  ↓
more pool pressure
  ↓
more timeouts
```

This is a classic cascading-failure pattern.

---

## 26. Connection Pool Sizing — Interview Answer

---

If asked:

```
"How do you decide HikariCP pool size?"
```

Don't say:

```
"I'll set it equal to CPU count."
```

A stronger answer:

```
I would size the pool based on database capacity, query latency, transaction duration,
application concurrency, and the number of application instances. The pool should be large
enough to keep the database utilized but not so large that concurrent queries create contention.
I would validate the chosen size with load testing and monitor active connections, pending
acquisition, query latency, and database CPU/IO. In a distributed deployment I would also
```

*calculate the aggregate maximum connections across all instances.*

That's a solid production-level answer.

---

## 27. Very Important: Connection Pool Sizing Across Instances

---

Suppose:

3 pods

Each has:

`maximumPoolSize = 20`

Potential maximum:

$3 \times 20 = 60$  connections

Scale to:

10 pods

Now:

$10 \times 20 = 200$

So Kubernetes autoscaling can unintentionally exhaust the DB connection limit if pool sizes aren't considered globally.

---

## 28. What Causes Hikari Pool Exhaustion?

---

Remember this list:

1. Slow SQL
2. Long transactions
3. DB lock contention
4. Connection leaks
5. Pool too small
6. Sudden traffic spike
7. Too many application instances
8. External calls while holding DB transaction/resources
9. Database itself is saturated

The pool being "full" is a **symptom**. Find out why connections are staying occupied.

---

## 29. HikariCP + N+1

---

This connects directly to your previous topic.

Suppose you have N+1:

```
1 parent query
+
1000 child queries
```

Each query needs DB access.

So N+1 can create:

```
more connection acquisition
more DB round trips
longer transactions
higher connection occupancy
```

Therefore:

```
N+1
↓
DB workload increases
↓
connections stay busy longer
↓
pool contention
```

This is why ORM performance problems can eventually appear as **connection-pool problems**.

---

## 30. HikariCP + Long Transactions

---

Consider:

```
@Transactional
public void process() {

    repository.findSomething();

    callHydra();      // 5 sec

    callAnotherApi(); // 3 sec

    repository.save(...);
}
```

Even though the actual DB operations may take milliseconds, the transaction can span external operations.

If the connection is held through that transaction:

```
Connection
↓
DB operation
↓
5-second HTTP call
↓
3-second HTTP call
↓
DB operation
```

That's wasteful.

A better design often separates:

DB transaction

from:

external network operation

where the business semantics permit it.

The exact behavior depends on transaction boundaries and when the persistence context actually obtains/holds the JDBC connection, but the general design principle remains important.

---

## 31. Interview Questions

---

### Fundamentals

1. What is connection pooling?
2. Why do we need connection pooling?
3. What is HikariCP?
4. Why is Hikari used in Spring Boot?
5. What is a `DataSource` ?

### Hikari

6. What does `maximumPoolSize` mean?
7. What is `minimumIdle` ?
8. What is `connectionTimeout` ?
9. What is `idleTimeout` ?
10. What is `maxLifetime` ?
11. What is leak detection?
12. What happens when the pool is exhausted?

### Architecture

13. Connection pool vs thread pool?
14. HikariCP vs HTTP connection pool?
15. Connection pooling vs transactions?
16. Does `connection.close()` physically close the connection?
17. How does Hibernate get a database connection?

### Production

18. How would you troubleshoot connection pool exhaustion?
  19. Why can increasing pool size make things worse?
  20. How do you size a pool in a microservice environment?
  21. Why can N+1 affect connection-pool utilization?
  22. How can long transactions cause pool starvation?
-

## 32. Interview-Ready Answer

---

If asked:

**“Explain HikariCP and connection pooling.”**

A strong answer is:

*HikariCP is a JDBC connection pool commonly used by Spring Boot applications. Instead of creating a physical database connection for every request, the pool maintains reusable connections. When Hibernate needs to execute SQL, it obtains a connection from the configured `DataSource`, which is typically backed by HikariCP, uses it, and returns it to the pool. Important settings include maximum pool size, connection acquisition timeout, idle timeout, and maximum connection lifetime. If all connections are busy, new requests wait for a connection and can eventually time out. Pool size should be based on database capacity, workload, transaction duration, and the total number of application instances rather than simply making the pool as large as possible.*

---

## 33. BPS-Specific Questions — Inspect Your Codebase

---

Now connect this to your actual BPS application.

Search configuration for:

```
spring.datasource
hikari
maximum-pool-size
minimum-idle
connection-timeout
idle-timeout
max-lifetime
```

Then determine:

### Database

- Which database does BPS use?
- Is HikariCP explicitly configured or relying on Spring Boot defaults?
- What’s the configured maximum pool size?

### Runtime behavior

- How many BPS application instances run simultaneously?
- Therefore, what is the **aggregate possible DB connection count**?

For example:

```
N instances × maxPoolSize
```

### Transactions

Inspect:

```
@Transactional
```

and ask:

- How long do the transactions typically remain open?
- Are Hydra/Chase calls made inside the transaction?
- Are there long-running queries?

## Production troubleshooting

If BPS shows:

```
Connection is not available, request timed out
```

be ready to explain:

```
Hikari pool exhaustion
↓
Check active/idle/waiting metrics
↓
Check slow SQL
↓
Check long transactions
↓
Check DB locks
↓
Check connection leaks
↓
Check traffic / number of instances
↓
Only then decide whether pool sizing is the actual problem
```

## Strong BPS interview scenario

**“BPS suddenly starts returning database connection timeout errors during a payment spike. How would you investigate it?”**

Your answer should connect:

```
Traffic
↓
Application threads
↓
Hikari pool
↓
Hibernate/JPA
↓
SQL latency / locks / N+1
↓
Database capacity
```

That ties **HikariCP + JPA/Hibernate + N+1 + transactions** together, which is exactly the kind of cross-topic reasoning expected in a backend interview.